have an important role to play for APIs that are designed to serve browser-based clients served from the same origin as the API. In chapter 6 you'll learn about alternatives to cookies that do not require the server to maintain any per-client state, and in chapter 9 you'll learn how to use capability URIs for a more RESTful solution.

Cookie-based sessions are so widespread that almost every web framework for any language has built-in support for creating such session cookies, and Spark is no exception. In this section you'll build a `TokenStore` implementation based on Spark's session cookie support. To access the session associated with a request, you can use the `request.session()` method:

```
Session session = request.session(true);
```

Spark will check to see if a session cookie is present on the request, and if so look up any state associated with that session in its internal database. The single `boolean` argument indicates whether you would like Spark to create a new session if one does not yet exist. To create a new session, you pass a `true` value, in which case Spark will generate a new session token and store it in its database. It will then add a Set-Cookie header to the response. If you pass a `false` value, then Spark will return `null` if there is no Cookie header on the request with a valid session token.

As we can reuse the functionality of Spark's built-in session management, the implementation of the cookie-based token store is almost trivial, as shown in listing 4.7. To create a new token, you can simply create a new session associated